

Advanced Time Series Forecasting with Attention Mechanisms

Project Overview

This project addresses the challenge of predicting complex, multivariate time series data. While standard recurrent models like LSTMs are effective, they can struggle with long-range dependencies in noisy datasets. This implementation introduces a Self-Attention layer to dynamically weight the importance of past time steps relative to the current forecast.

The repository includes:

- **Synthetic Multivariate Data Generation:** Simulates complex, correlated real-world dependencies.
- **Hybrid LSTM-Attention Architecture:** A custom PyTorch model combining temporal feature extraction with attention-based context weighting.
- **Baseline Comparison:** A standard LSTM model used to evaluate the performance gains of the attention mechanism.

Key Features

1. Custom Attention Mechanism

The AttentionBlock class computes importance scores for each hidden state in the LSTM sequence.

- **Dynamic Weighting:** Uses a series of Linear layers and a Softmax activation to assign weights.
- **Context Vector:** Aggregates the weighted hidden states into a single context vector for the final prediction.

2. Model Architectures

- **TimeSeriesModel:** A stacked LSTM (2 layers) followed by the custom Attention layer and a fully connected output head.
- **BaselineLSTM:** A traditional LSTM setup that uses only the final hidden state for prediction.

3. Rigorous Evaluation

The project measures performance using Root Mean Squared Error (RMSE) to compare the predictive accuracy of the two models.

Technical Specifications

Parameter	Value
Framework	PyTorch
Input Features	5 (Multivariate)
Sequence Length	24 steps
Hidden Dimensions	64
Optimization	Adam Optimizer
Loss Function	MSE Loss

Implementation Guide

Prerequisites

bash

pip install torch numpy pandas matplotlib scikit-learn

Usage

The core logic is contained in lstm.py. To run the training and evaluation pipeline:

bash

python [lstm.py](#)

Code Structure

- `generate_multivariate_data()`: Creates a synthetic dataset with sine/cosine waves and random noise to simulate real-world signals.
- `create_sequences()`: Transforms raw data into a supervised learning format.
- `train_model()`: A modular training loop that handles backpropagation and loss calculation.

Results and Analysis

The output of the script provides a direct comparison of RMSE values. Typically, the Attention-based model captures non-linear dependencies more effectively than the baseline, leading to a lower error rate in complex forecasting scenarios.

For production-quality deployment, ensure the data is modularized and include docstrings for all custom layers.